Chapter 6: Modular Programming

Mohamed M. Sabry Aly
N4-02c-92

1

Learning Objectives

- Describe ARM instructions in implement subroutines
- Describe passing parameters to subroutines using:
  - Registers
  - Memory

2

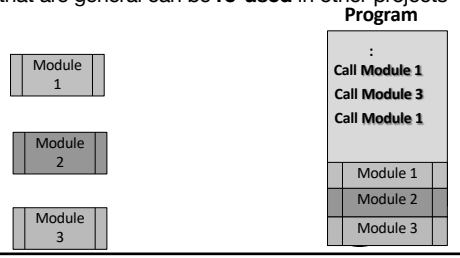
Modular Program Design

- Real-world applications are very large and complex
  - Google Chrome: ~6.7 Millions line of code
  - Android OS: 12-15 Millions line of code
  - Boeing 787: ~6.5 Millions line of code
- Large software cannot be a single function
- It is decomposed into several less complex **modules**

3

Modular Program Design

- Software decomposed to several less complex **modules**
  - Modules can be designed and tested **independently**
  - Modules can reduce overall **program size**
    - Same module may be required in several places
  - Modules that are general can be **re-used** in other projects



4

Characteristics of a Good SW Module

- Loose coupling—**data** within module is entirely **independent** of other modules (local variables)
- Strong modularity—should perform a **single** logically coherent **task**

5

Example: Standard Deviation

Case 1: all in one c main()

$$\sigma = \sqrt{\frac{\sum(x_i - \mu)^2}{N}}$$

```
int main() {
    .
    float avg=0.0; //initialization
    for(int i=0;i<N;i++){
       avg +=x[i];
    }
    avg/=N;
    float sigma = 0.0;
    for(int i=0;i<N;i++){
       sigma +=pow(x[i]-avg),2);
    }
    sigma/=N;
    sigma=sqrt(sigma);
    return 0;
}
```

6

6

Example: Standard Deviation

Case 2: Modular

$$\sigma = \sqrt{\frac{\sum(x_i - \mu)^2}{N}}$$

```
int main() {
    …
    float avg = sum(x,N)/N;
    float sigma = Sigma_f(x,avg,N);
    return 0;
}

float sum( int* x, int N){
    float tot=0.0; //initialization
    for(int i=0;i<N;i++){
       tot +=x[i];
    }
    return tot;
}

float Sigma_f( int* x, float avg, int N){
    float sigma = 0.0;
    for(int i=0;i<N;i++){
       sigma +=pow(x[i]-avg),2);
    }
    sigma/=N;
    sigma=sqrt(sigma);
    return sigma;
}
```

7

7

Example: Standard Deviation

Case 2: Modular

How will this be translated to assembly instructions?

How will ARM go to these two functions and return?

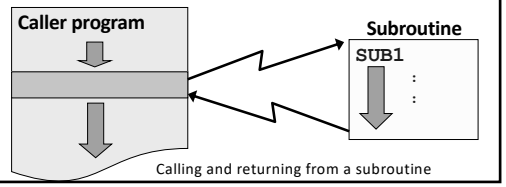
```
int main() {
    …
    float avg = sum(x,N)/N;
    float sigma = Sigma_f(x,avg,N);
    return 0;
}

float sum( int* x, int N){
    float tot=0.0; //initialization
    for(int i=0;i<N;i++){
       tot +=x[i];
    }
    return tot;
}

float Sigma_f( int* x, float avg, int N){
    float sigma = 0.0;
    for(int i=0;i<N;i++){
       sigma +=pow(x[i]-avg),2);
    }
    sigma/=N;
    sigma=sqrt(sigma);
    return sigma;
}
```

8

8

Subroutines

- Modules (e.g., functions in C) are implemented as **subroutines**
- Subroutine can be called from various parts of the program
- Caller and callee
  - Caller: the program that calls subroutine (SUB1)
  - Callee: subroutine (SUB1)

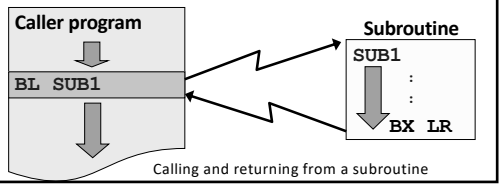


9

Subroutines

- On completion, subroutine returns control to the caller
  - Exactly after the subroutine was called
- Calling and returning from a subroutine
  - To go to subroutine (SUB1): **`BL SUB1`**
  - To return to caller program: **`BX LR`**

`BL: branch with link`
`BX: branch and exchange`



10

Why `BL` and `BX` and not `B`?

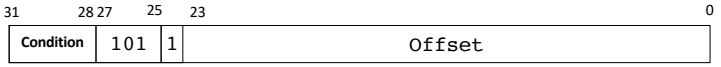
- Main program branches to subroutine:
  - Can be done with B ➔ what is the draw back?
  - B overwrites value in PC ➔ oldPC value in main program is LOST
  - Maybe add another branch at the end of subroutine ➔ need to know the exact mem location during compilation, not an effective approach

11

11

Branch with Link (`BL`)

- **`BL`** used to make subroutine call
- Return address (PC contents + 4) is stored in the link register (R14)



- We can also conditionally make a functional call (more of this later)

12

Branch with Link (BL)

- **BL** used to make subroutine call

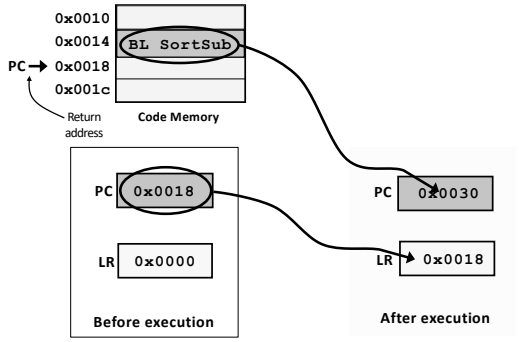
```
Subroutine Call
BL SortSub
```

- Execution sequence:
- Return address (PC contents + 4) is stored in the link register (R14)
- The subroutine address is stored to the PC

13

BL (Execution example)

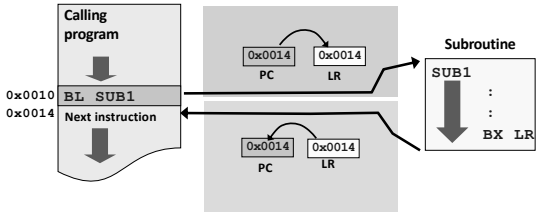
- BL SortSub (assume SortSub is in 0x0030)



14

BX instruction

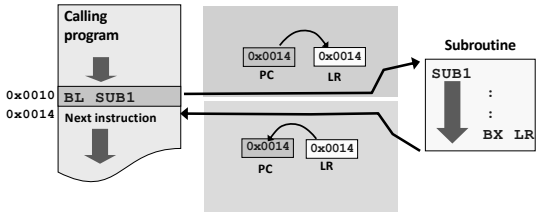
- `BX lr` returns from subroutine
- `lr` contains the return address, the instruction copies the value over to `PC`



15

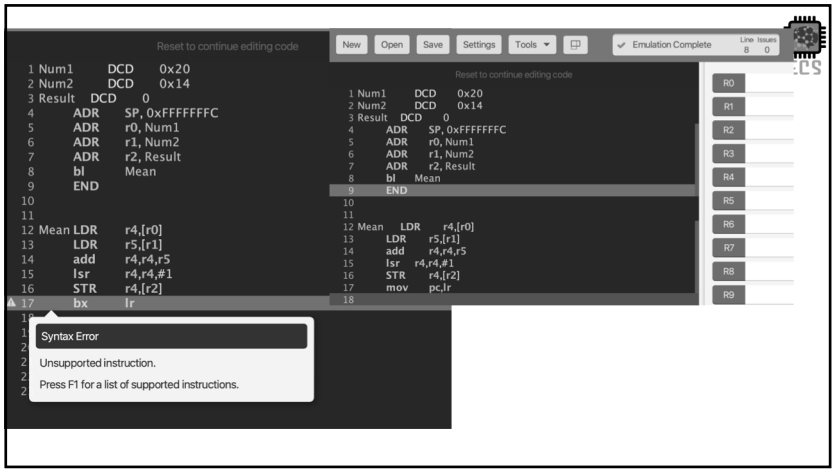
BX instruction

- `BX lr` returns from subroutine
- `lr` contains the return address, the instruction copies the value over to `PC`



- Note: VisUAL does not support `bx lr`
  - Use `MOV PC, LR`

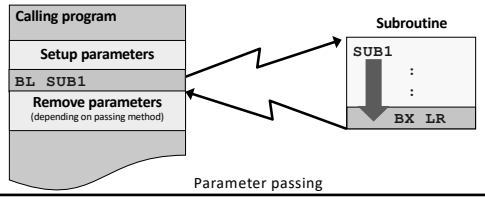
16



17

Parameter Passing

- Calling programs need to pass parameters to influence a subroutine's execution.
- Parameters must be setup properly before the subroutine is called and appropriately removed after returning.
- There are three basic methods to pass parameters, via **registers**, **memory block** or the **system stack**.



18

18

Parameter Passing using Registers

- Parameters are placed into the register before calling the subroutine
- **Number** of parameters passed are **limited** to the **available registers**
  - Useful when number of parameters are **small**
  - Not all **R0-R12** are preferred to pass parameters

19

Parameter Passing using Registers

- Parameters are placed into the register before calling the subroutine
- **Number** of parameters passed are **limited** to the **available registers**
  - Useful when number of parameters are **small**
  - Not all **R0-R12** are preferred to pass parameters

Calling convention

R0-R3	**R4-R11**
Can be used to pass argument values Also used to return values from subroutine Subroutine can modify values	Used to hold local variables Not for passing arguments Must be preserved in the subroutine

20

Parameter Passing using Registers

- Parameters are placed into the register before calling the subroutine
- **Number** of parameters passed are **limited** to the **available registers**
  - Useful when number of parameters are **small**
  - Not all **R0-R12** are preferred to pass parameters

Calling convention

R0-R3	**R4-R11**
Can be used to pass argument values to return values from subroutine Subroutine can modify values	Used to hold local variables Not for passing arguments Must be preserved in the subroutine

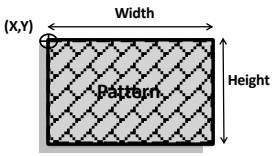
R12: Scratchpad register, does not need to be preserved
Can be used sometimes as return register

21

Parameter Passing using Registers

- Parameters are placed into the register before calling the subroutine
- **Number** of parameters passed are **limited** to the **available registers**
  - Useful when number of parameters are **small**
  - Not all **R0-R12** are preferred to pass parameters

```
Draw_rectangle(
X, Y,
Height, Width,
Border, Line_style
Fill, Color, Pattern,
Shadow);
```



22

22

Parameter Passing using Registers

- Parameters are placed into the register before calling the subroutine
- **Number** of parameters passed are **limited** to the **available registers**
  - Useful when number of parameters are **small**
  - Not all **R0-R12** are preferred to pass parameters

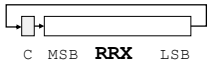
- **Pro** – **efficient** as parameters are already in register within the subroutine and can be used immediately.
- **Con** – **lacks generality** due to the limited number of registers.

23

23

Example: Bit Counting Subroutine

- Write a subroutine to:
  - Count the number of "1" bits in a word.
  - Return result in register **R0**.

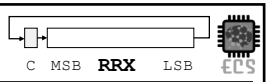


- **Design considerations:**
- How do we transfer the word into the subroutine?
  - Put the word into a **register**, which can then be accessed within the subroutine (e.g. register **R1**).
- How do we check if each individual bit is a "1" or a "0"?
  - Rotate **R1** right 32 times with the carry bit. After each rotate, test carry bit to check if (**C**=1). If yes, increment bit counter register **R0**.

24

24

Solution #1




```
Count1s
```

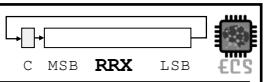
Start by labeling the subroutine
and placing the return instruction

```
        MOV     PC, LR        ; same as bx lr
```

Value passed in R1, return value in R0 VisUAL does not support `bx lr`

25

Solution #1




```
Count1s  MOV     R0, #0   ;Clear R0
         MOV     R2, #32  ; Set counter R2 with 32
```

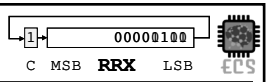
Initialize R0 with 0 and
the counter register R2 with 32

```
         MOV     PC, LR        ; same as bx lr
```

Value passed in R1, return value in R0 VisUAL does not support `bx lr`

26

Solution #1




```
Count1s  MOV     R0, #0  ;Clear R0
         MOV     R2, #32 ; Set counter R2 with 32
Loop     RRXS    R1,R1   ; Rotate right and extend R1
```

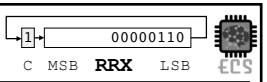
Rotate to the right (arithmetic), and use the carry
S: set the status registers after rotation

```
         MOV     PC, LR        ; same as bx lr
```

Value passed in R1, return value in R0 VisUAL does not support `bx lr`

27

Solution #1




```
Count1s  MOV     R0, #0        ;Clear R0
         MOV     R2, #32       ; Set counter R2 with 32
Loop     RRXS    R1,R1         ; Rotate right and extend R1
         ADC     R0,R0,#0      ; Add the carry to R0
```

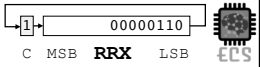
Add the carry value to R0

```
         MOV     PC, LR        ; same as bx lr
```

Value passed in R1, return value in R0 VisUAL does not support `bx lr`

28

Solution #1




```
Count1s  MOV     R0, #0        ; Clear R0
         MOV     R2, #32       ; Set counter R2 with 32
Loop     RRXS    R1,R1         ; Rotate right and extend R1
         ADC     R0,R0,#0      ; Add the carry to R0
         SUBS    R2,R2,#1      ; decrement counter by 1
         BNE     Loop          ; loop if not zero
         MOV     PC, LR        ; same as bx lr
```

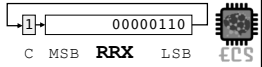
Decrement and set Status registers

Value passed in R1, return value in R0

VisUAL does not support `bx lr`

29

Solution #1




```
Count1s  MOV     R0, #0        ;Clear R0
         MOV     R2, #32  ; Set counterR2 with 32
Loop     RRXS    R1,R1         ; Rotate right and extend R1
         ADC     R0,R0,#0      ; Add the carry to R0
         SUBS    R2,R2,#1      ; decrement counter by 1
         BNE     Loop          ; loop if not zero
         MOV     PC, LR        ; same as bx lr
```

Issue, the carry of SUBS
Will be used in RRXS → R1 value is destroyed

Value passed in R1, return value in R0

VisUAL does not support `bx lr`

30

Solution #2

```
Count1s  EOR     R0,R0,R0      ;Clear R0
         ADD     R2, R0, #32   ; Set counterR2 with 32
         ADD     R3, R0, #1    ; Set R3 with 1
Loop     AND     R4, R3, R1, ROR R2        ; ?
         ADD     R0,R0,R4      ; Add the lsb of R0
         SUBS    R2,R2,#1      ; decrement counter by 1
         BNE     Loop          ; loop if not zero
         MOV     PC, LR        ; same as bx lr
```

Value passed in R1, return value in R0

VisUAL does not support `bx lr`

31

Solution #2

```
Count1s  EOR     R0,R0,R0      ;Clear R0
         ADD     R2, R0, #32   ; Set counterR2 with 32
         ADD     R3, R0, #1    ; Set R3 with 1
Loop     AND     R4, R3, R1, ROR R2        ; ?
         ADD     R0,R0,R4      ; Add the lsb of R0
         SUBS    R2,R2,#1      ; decrement counter by 1
         BNE     Loop          ; loop if not zero
         MOV     PC, LR        ; same as bx lr
```

Value passed in R1, return value in R0

VisUAL does not support `bx lr`

32

Solution #2

```
Count1s  EOR      R0,R0,R0      ;Clear R0
         ADD      R2, R0, #32   ; Set counterR2 with 32
         ADD      R3, R0, #1    ; Set R3 with 1
Loop     AND      R4, R3, R1, ROR R2         ; ?
```

Set R3 as a mask with a value of 1
Then apply this mask with a rotated value of R1

```
         MOV      PC, LR        ; same as bx lr
```

Value passed in R1, return value in R0 VisUAL does not support `bx lr`

33

Solution #2

```
Count1s  EOR      R0,R0,R0      ;Clear R0
         ADD      R2, R0, #32   ; Set counterR2 with 32
         ADD      R3, R0, #1    ; Set R3 with 1
Loop     AND      R4, R3, R1, ROR R2         ; ?
```

Set R3 as a mask with a value of 1
Then apply this mask with a rotated value of R1
The **rotated R1 is not stored anywhere**

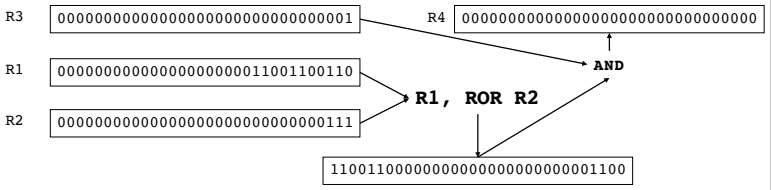
```
         MOV      PC, LR        ; same as bx lr
```

Value passed in R1, return value in R0 VisUAL does not support `bx lr`

34

A Deep Dive

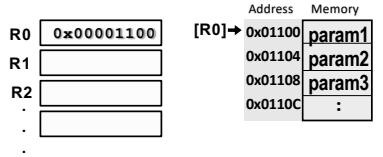
```
AND     R4, R3, R1, ROR R2
```



35

Parameter Passing using Memory

- A region in memory is treated like a mailbox and is used by both the calling program and subroutine.
  - Parameters to be passed are gathered into a **block** at a predefined memory location
  - The start address of the memory block is passed to the subroutine via an **address register**.
  - Useful for passing **large number of parameters**.



36

36

Example: Lower to Upper Case Subroutine

- Write a subroutine to:
  - To convert an ASCII string from lower to upper case.
  - The string is terminated by a NULL character (**0x00000000**).
  - The start address of the string is passed via **R0**.
  - A segment of the calling program shows how the parameter is setup and the subroutine called:

```
;Calling program
   :
MOV   R0,#0x100   ;move start addr. of string to R1
BL    Lo2Up       ;branch to Lo2Up subroutine
   :
```

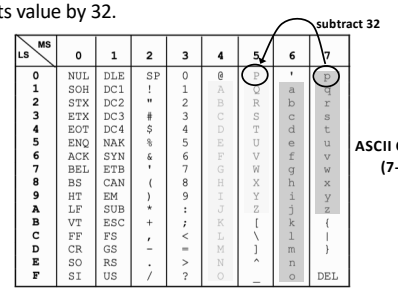
Address	Memory
0x100	"a"
0x104	"p"
0x108	"p"
0x10C	"l"
0x110	"e"
0x114	0x000
0x118	:

37

37

Algorithm Design

- How to convert an ASCII character from lower to upper case?
  - Check that the character's value is between 'a' and 'z'.
  - If so, subtract its value by 32.



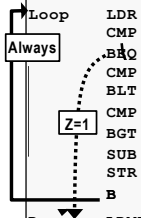
MS / LS	0	1	2	3	4	5	6	7
0	NUL	DLE	SP	0	@	P	`	p
1	SOH	DC1	!	1	A	Q	a	q
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3	#	3	C	S	c	s
4	EOT	DC4	$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB	'	7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	HT	EM	)	9	I	Y	i	y
A	LF	SUB	*	:	J	Z	j	z
B	VT	ESC	+	;	K	[	k	{
C	FF	FS	,	<	L	\	l	|
D	CR	GS	-	=	M	]	m	}
E	SO	RS	.	>	N	^	n	
F	SI	US	/	?	O	_	o	DEL

ASCII Character Set (7-Bit Code)

38

38

Possible Solution




```
Lo2Up   STMFD   SP!,{r0,r1}   ;save registers used within subroutine
Loop    LDR     R1,[R0],#4    ;get current char from string in memory
        CMP     R1,#0         ;Compare with NULL
        BEQ     Done          ;if NULL char, branch to Done
        CMP     R1,#0x061     ;compare with lower limit "a"
        BLT     Loop          ;if smaller than "a", do not convert
        CMP     R1,#0x07A     ;compare with upper limit "z"
        BGT     Loop          ;if greater than "z" do not convert
        SUB     R1,R1,#32     ;convert to upper case by subtracting 32
        STR     R1,[R0,#-4]   ;write modified char back to string in memory
        B       Loop          ;branch back to Loop
Done    LDMFD   SP!,{r0,r1}   ;restored saved registers before returning
        MOV     PC,LR         ;return from subroutine
```

Note: Subroutine modifies **R0** & **R1** but restores their original contents before returning. This produces a **transparent subroutine** that does not effect the proper operation of calling program

39

39

Possible Solution

```
Lo2Up   STMFD   SP!,{r0,r1}   ;save registers used within subroutine
Loop    LDR     R1,[R0],#4    ;get current char from string in memory
        Increment R0 by 4 and update R0
        (to point to next memory location)
        Refer to the location to R0-4 (the original R0)
        STR     R1,[R0,#-4]   ;write modified char back to string in memory
        B       Loop          ;branch back to Loop
Done    LDMFD   SP!,{r0,r1}   ;restored saved registers before returning
        MOV     PC,LR         ;return from subroutine
```

Note: Subroutine modifies **R0** & **R1** but restores their original contents before returning. This produces a **transparent subroutine** that does not effect the proper operation of calling program

40

40

Possible Solution

```
Lo2Up   STMFD   SP!,{r0,r1}   ;save registers used within subroutine
                                  → R0=0x100
Loop    LDR     R1,[R0],#4    ;get current char from string in memory
        Increment R0 by 4 and update R0
        (to point to next memory location)
                                  → R0=0x104
                                  → R0-4=0x100
        Refer to the location to R0-4 (the original R0)
        STR     R1,[R0,#-4]   ;write modified char back to string in memory
        B       Loop          ;branch back to Loop
Done    LDMFD   SP!,{r0,r1}   ;restored saved registers before returning
        MOV     PC,LR         ;return from subroutine
```

Note: Subroutine modifies **R0** & **R1** but restores their original contents before returning. This produces a **transparent subroutine** that does not effect the proper operation of calling program

41

41

Optimizing Memory Usage

- Example assumes each character in 32 bits
  - But each character requires 8 bits only
- Can we access each Byte separately?

Address	Memory
0x100	"a"
0x101	"p"
0x102	"p"
0x103	"l"
0x104	"e"
0x105	0x000
0x106	:

42

42

Optimizing Memory Usage

- Example assumes each character in 32 bits
  - But each character requires 8 bits only
- Can we access each Byte separately?

Use the {B} option in Access

LDRB
STRB

Address	Memory
0x100	"a"
0x101	"p"
0x102	"p"
0x103	"l"
0x104	"e"
0x105	0x000
0x106	:

43

43

Efficient Memory Solution

```
Lo2Up   STMFD   SP!,{r0,r1}   ;save registers used within subroutine
Loop    LDRB    R1,[R0],#1    ;get current char from string in memory
        CMP     R1,#0         ;Compare with NULL
        BEQ     Done          ;if NULL char, branch to Done
        CMP     R1,#0x061     ;compare with lower limit "a"
        BLT     Loop          ;if smaller than "a", do not convert
        CMP     R1,#0x07A     ;compare with upper limit "z"
        BGT     Loop          ;if greater than "z" do not convert
        SUB     R1,R1,#32     ;convert to upper case by subtracting 32
        STRB    R1,[R0,#-1]   ;write modified char back to string in memory
        B       Loop          ;branch back to Loop
Done    LDMFD   SP!,{r0,r1}   ;restored saved registers before returning
        MOV     PC,LR         ;return from subroutine
```

Note: Subroutine modifies **R0** & **R1** but restores their original contents before returning. This produces a **transparent subroutine** that does not effect the proper operation of calling program

44

44

Summary

- `BL` is required to call a subroutine, return address is in `LR` register
- A return from subroutine is done with `BX LR` or `MOV PC,LR`
- Passing parameters **using registers** is the simplest and fastest.
  - Number of parameters that can be passed is **limited** by the **available registers**.
- Passing parameters using a **memory block** can support a **large number** of parameters or data types like arrays.

45